MACHINE PREDICTIVE MAINTENANCE USING MACHINE LEARNING

Submitted in partial fulfilment of the requirement

for the award of the degree of

BACHELOR'S OF COMPUTER APPLICATION

TO

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY

Guide:

Dr. Ramandeep Kaur

(Associate Professor)

Submitted by:

Vansh Arora

00321102022



INSTITUTE OF INFORMATION TECHNOLOGY & MANAGEMENT

NEW DELHI- 110058

BATCH (2022-2025)

CERTIFICATE

I, (**Vansh Arora 00321102022**) certify that the Major Project Report (BCA-308) entitled "**Creating Machine Predictive Maintenance Using Machine Learning**" is done by me and it is an authentic work carried out by me at **Institute of Information Technology & Management**. The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of the Student

Date:

Certified that the Project Report (BCA-308) entitled "**Creating Machine Predictive Maintenance Using Machine Learning**" done by the above student is completed under my guidance.

Signature of the Guide:

Date:

Name of the Guide: Dr Ramandeep Kaur

Designation: Associate Professor

Prof. (Dr.) Ganesh Kumar Wadhwani
Counter Sign HOD- Computer Science

Prof. (Dr.) Rachita Rana
Counter Sign Director

ACKNOWLEDGEMENT

I want to express my sincere gratitude to everyone who helped to finish this data science and machine learning project. I want to start by expressing my gratitude to Dr Ramandeep Kaur for all her help and support during this educational journey. His extensive experience and profound understanding have greatly influenced the course and result of this project.

I would like to extend my sincere gratitude to Dr. Ganesh Kumar Wadhwani for providing me with an outstanding training program that enabled an all-encompassing educational experience. My ability to successfully apply theoretical information to real-world situations was greatly aided by the tools and learning environment that were made available.

I am grateful to my classmates who have offered their support and encouragement, contributing to a collaborative and stimulating learning environment. The exchange of ideas and constructive discussions have greatly enriched my project.

Thank you all for your invaluable contributions to this project.

Vansh Arora

(00321102022)

List of Tables

Table No.	Table Description	Page No.
1.1	Hardware Specification Tools	4
1.2	Software Specification Tools	4
2.1	Literature Summary	12-15
4.1	Hardware requirement for Jupyter Notebook	18
4.2	Software Requirement for Jupyter Notebook	18

List of Figures

Figure No.	Figure Description	Page No.
1.1	PERT Chart	8
4.1	CSV File Uploadation	19
4.2	Importing Libraries	19
4.3	Importing Dataset	19
4.4	Checking Outliers Through Box Plot	25
4.5	Boxplot After Removing Outliers	26
4.6	Checking Correlation Through Heatmap	37
4.7	Classification Report of Logistic Regression	38
4.8	Accuracy of Decision Tree Algorithm	38
4.9	Accuracy of Random Forest Algorithm	38
4.10	Accuracy of K-Nearest Neighbor Algorithm	38
4.11	Accuracy of Naïve Bayes Algorithm	39
4.12	Accuracy of Support Vector Machine Algorithm	39
4.13	Comparison of Accuracy	39
4.14	GUI using Streamlit	40
4.15	Prediction of Failures	41

TABLE OF CONTENTS

S.No.	TOPIC	PAGE No.
1.	CERTIFICATE	-
2.	ACKNOWLEDGEMENT	-
3.	ABSTRACT	1
4.	SYNOPSIS	2-4
5.	CHAPTER-1 INTRODUCTION	
	1.1 Description of Topic	5
	1.2 Problem Statement	6
	1.3 Objectives	6-7
	1.4 Scope of Project	7
	1.5 Project Planning Activities	
	1.5.1 Team Member Wise Work Distribution Table	8
	1.5.2 PERT Chart	8
	1.6 Organization of Report	8-10
6.	CHAPTER-2 LITERATURE REVIEW	11-15
7.	CHAPTER-3 SYSTEM DESIGN AND METHODOLOGY	
	3.1 System Design	16
	3.2 Algorithm Used	16-17
8.	CHAPTER-4 IMPLEMENTATION AND RESULT	
	4.1 Hardware and Software Requirement	18
	4.2 Implementation Details	19-36
	4.3 Results	37-41
9.	CHAPTER-5 CONCLUSION AND FUTURE WORK	
	5.1 Conclusion	42
	5.2 Future Scope	42-43
10.	References	44-45

ABSTRACT

Predictive maintenance has emerged as a vital strategy in industrial systems to reduce unexpected equipment failures, minimize downtime, and optimize maintenance schedules. This project presents a machine predictive maintenance solution leveraging supervised machine learning algorithms to forecast potential failures based on historical sensor data. Using a labeled dataset comprising features such as torque, temperature, pressure, and usage cycles, several supervised classification models—including Random Forest, Support Vector Machine (SVM), and Logistic Regression—were trained and evaluated. The Random Forest model delivered the highest accuracy and robustness, making it the final choice for deployment.

To enhance accessibility and user interaction, a graphical user interface (GUI) was developed using Streamlit, an open-source Python framework for building data-driven web applications. The GUI allows users to upload new data in real time, view predictive results, and visualize critical metrics such as feature importance, prediction probability, and maintenance trends. This interface also supports basic preprocessing options, enabling end users with minimal technical background to interact with the predictive system efficiently.

By integrating machine learning with an intuitive web-based GUI, the project demonstrates a scalable and user-friendly approach to predictive maintenance. The system not only aids in early detection of potential failures but also empowers maintenance teams with actionable insights, contributing to smarter and more cost-effective operations. The modularity of the system also ensures adaptability across different types of machinery and industrial contexts.

SYNOPSIS

Title of Project

The Title of Project is “Machine Predictive Maintenance using Machine Learning”.

Problem Definition

In industrial environments, unexpected machine failures can lead to significant operational downtime, increased maintenance costs, and production delays. Traditional maintenance strategies such as reactive (fix after failure) or preventive (scheduled servicing) approaches are either too late or not cost-effective, as they do not account for the actual condition of the machinery. This creates a critical need for a smarter solution that can predict failures in advance and recommend timely interventions.

The core problem addressed in this project is: "How can we use historical machine data and supervised machine learning algorithms to accurately predict potential machine failures and present these predictions in an accessible, interactive format for end-users without technical expertise?"

Significance

The significance of this project lies in its potential to transform traditional maintenance practices through the integration of machine learning and modern web technologies. By predicting equipment failures before they occur, organizations can avoid costly downtime, improve asset lifespan, and enhance overall operational efficiency. The use of supervised learning algorithms ensures that the predictions are data-driven and continuously improvable as more machine data becomes available. Moreover, the development of a user-friendly Streamlit-based GUI makes advanced predictive analytics accessible to non-technical users, such as maintenance staff and plant operators. This democratization of AI-powered tools fosters smarter decision-making on the shop floor and bridges the gap between complex data science models and real-world industrial applications. Ultimately, the system contributes to more proactive, reliable, and cost-effective maintenance strategies in smart manufacturing environments.

Objective

The primary objective of this project is to develop an intelligent predictive maintenance system using supervised machine learning algorithms to anticipate potential machine failures based on historical sensor data. The project aims to evaluate and compare various classification models, such as Random Forest, Support Vector Machine (SVM), and Logistic Regression, to determine the most accurate and reliable approach for failure prediction. Another key objective is to preprocess and analyze the input data to ensure it is suitable for training effective machine learning models. To make the system accessible to non-technical users, the project includes the development of an interactive and user-friendly Graphical User Interface (GUI) using Streamlit. This GUI allows users to upload data, visualize key performance indicators, and receive real-time predictions regarding machine health. Overall, the project strives to bridge the gap between advanced machine learning techniques and practical maintenance decision-making, promoting cost savings, reduced downtime, and improved operational efficiency in industrial environments.

Scope

This project focuses on the development and deployment of a predictive maintenance system using supervised machine learning techniques, with an emphasis on real-time usability through a web-based interface. The system is designed to work with historical and real-time sensor data collected from industrial machines, such as temperature, pressure, vibration, and operational cycles, to predict the likelihood of equipment failure. The scope includes data preprocessing, model training and evaluation, and the creation of an interactive GUI using Streamlit that allows users to input new data and receive instant predictions. While the current implementation is trained on a specific dataset, the system is designed to be modular and scalable, allowing adaptation to various types of industrial machinery and environments with similar data structures. However, the scope does not include integration with live industrial IoT systems or physical machine control, and the accuracy of predictions depends on the quality and representativeness of the input data. The project serves as a prototype that demonstrates the potential for AI-driven predictive maintenance in real-world applications.

Hardware Specification Tools

Minimum Hardware Requirements	
Processor	Intel® Core™ i5
CPU	1.60GHZ
Memory	At least 2.00 GB
Hard Disk	500 MB – 1GB
Display	Super VGA (1336 x 768) or higher-resolution monitor with 256 colours
Input Devices	Keyboard, Mouse

Table 1.1 Hardware Specification

Software Specification Tools

Minimum Software Requirements	
Front End	Python
Browser	Chrome (Recommended), Mozilla, etc
Development Tool	Jupyter Notebook, Google Collab, Anaconda

Table 1.2 Software Specification

Data Collection and Methodology

Data Collection is one of the most important aspects in Data Analysis. The dataset can be been taken from www.kaggle.com, www.brightdata.com, etc. Due to the wide adoption of machine learning models, simply having large datasets on a domain specific task does not ensure superior performance. Therefore, the dataset has to be cleaned and pre-processed before training. As Machine Learning models learn from the data, they are trained with automatic predictions are likely to mirror the human disagreement identified during annotation. As a result, having proper data cleaning and preprocessing of dataset is required. Tagging of data or annotation usually comes at the cost of having it to be manually annotated. It is a pretty time-consuming task. The dataset should be labelled and pre-processed before giving to Machine Learning models. The labelled sentiment data is used to measure the accuracy of the model by comparing correctly classified. verses falsely classified data which further helps to tune the ML model's performance.

CHAPTER-1

INTRODUCTION

1.1 Description of Topic

Predictive maintenance is a modern maintenance strategy that uses data analytics and machine learning to predict equipment failures before they occur. This proactive approach helps industries avoid unexpected downtime, reduce maintenance costs, and increase the lifespan of machinery. Traditional maintenance methods, such as reactive maintenance (repair after failure) or preventive maintenance (scheduled servicing), are either too late or too frequent, often leading to unnecessary costs or sudden breakdowns. Predictive maintenance offers a smarter alternative by analyzing patterns in machine sensor data to forecast potential issues in advance.

The focus of this project is to design and implement a predictive maintenance system using supervised machine learning algorithms. Historical data from machines, including key operational parameters like temperature, pressure, vibration, and run-time cycles, is used to train classification models that can identify early signs of machine failure. Several algorithms—such as Random Forest, Support Vector Machine (SVM), and Logistic Regression—are explored and compared to determine the best-performing model in terms of accuracy, precision, and recall.

To ensure that the system is practical and user-friendly, a Graphical User Interface (GUI) is developed using **Streamlit**, a Python-based web application framework. This GUI enables users, including maintenance engineers and plant operators with minimal technical expertise, to interact with the model through a web browser. Users can upload new sensor data, receive real-time predictions on machine health, and visualize critical information such as prediction probabilities and key feature influences. The GUI streamlines interaction with the model and helps bridge the gap between advanced analytics and real-world decision-making.

In summary, the project demonstrates how machine learning can be effectively combined with modern web technologies to build an intelligent, accessible, and scalable predictive maintenance solution that can be tailored to different industrial applications.

1.2 Problem Statement

In industrial operations, unexpected machine failures can lead to significant financial losses, operational inefficiencies, and production delays. Traditional maintenance strategies, such as reactive maintenance (addressing issues post-failure) and preventive maintenance (scheduled maintenance regardless of actual need), often result in either excessive downtime or unnecessary maintenance activities, neither of which optimizes operational performance. This inefficiency highlights the critical need for a more predictive approach to maintenance that can accurately forecast machine failures and allow for timely, targeted interventions.

The challenge lies in developing a predictive maintenance model that can effectively analyse large volumes of complex machine data to predict failures with high accuracy. The model must be robust, scalable, and capable of generalizing across different types of machinery and operational environments.

The problem is further compounded by the diverse factors that influence machine health, such as varying operating conditions, environmental factors, and the interactions between different machine components. Addressing this problem requires not only advanced data analysis techniques but also careful feature selection and model tuning to ensure that the predictions are both accurate and actionable. The ultimate goal is to reduce unplanned downtimes, optimize maintenance schedules, and improve the overall efficiency and reliability of industrial operations through predictive maintenance.

1.3 Objectives

The objective of this study is to develop a logistic regression model for machine predictive maintenance in forecasting potential machine failures. This involves:

1. **Data Collection and Preparation:** Gathering and preprocessing historical data, including sensor readings, operational metrics, and failure logs, to create a comprehensive dataset for model training and validation.
2. **Feature Engineering:** Identifying and selecting relevant features that influence machine health and failure, ensuring the model incorporates the most significant predictors.

3. **Model Development:** Designing and fine-tuning the logistic regression model to accurately predict the likelihood of machine failures, employing techniques such as parameter tuning and regularization to enhance model performance.
4. **Model Validation:** Evaluating the model's performance using metrics such as accuracy, precision, recall, and F1-score, and validating its effectiveness through rigorous testing and cross-validation.
5. **Practical Application and Impact Assessment:** Analysing the impact of the predictive maintenance model on reducing unplanned downtimes, optimizing maintenance schedules, and lowering overall maintenance costs, while exploring its applicability across different types of machinery and industries.

By achieving these objectives, the study aims to provide a reliable predictive maintenance solution that enhances operational efficiency and reduces costs associated with machine failures.

1.4 Scope of the Project

This project focuses on the development and deployment of a predictive maintenance system using supervised machine learning techniques, with an emphasis on real-time usability through a web-based interface. The system is designed to work with historical and real-time sensor data collected from industrial machines, such as temperature, pressure, vibration, and operational cycles, to predict the likelihood of equipment failure. The scope includes data preprocessing, model training and evaluation, and the creation of an interactive GUI using Streamlit that allows users to input new data and receive instant predictions. While the current implementation is trained on a specific dataset, the system is designed to be modular and scalable, allowing adaptation to various types of industrial machinery and environments with similar data structures. However, the scope does not include integration with live industrial IoT systems or physical machine control, and the accuracy of predictions depends on the quality and representativeness of the input data. The project serves as a prototype that demonstrates the potential for AI-driven predictive maintenance in real-world applications.

1.5 Project Planning Activities

1.5.1 Team Members wise Work Distribution

This project was undertaken by me, Vansh Arora. I managed all aspects of the project, from defining the statement of purpose to implementing the solution. The initial task involved clarifying the objectives and significance of the project, which was crucial for outlining the future steps and direction. I meticulously described the datasets used, which in turn informed the choice of methods and techniques applied.

The project involved several key phases, including data preprocessing, feature engineering, and the development of machine learning models. I handled all these tasks independently, ensuring each phase was executed with precision and attention to detail. Additionally, I planned the future steps for further development and optimization of the predictive maintenance model. Throughout the project, I employed efficient time management and thoughtful problem-solving to achieve optimal solutions.

The successful completion of the project reflects a high level of individual effort and effective utilization of ideas and methods to address the project's challenges and objectives.

1.5.2 PERT Chart

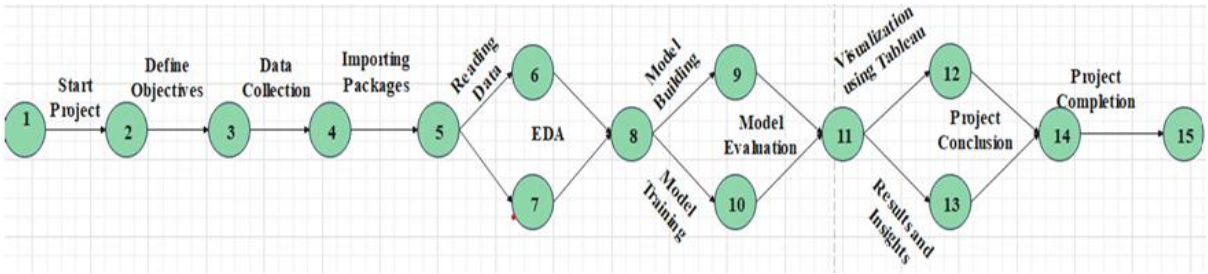


Fig.1.1 PERT Chart

1.6 Organization of Report

Chapter 1 - Introduction

- The project focuses on predicting machine failures using historical sensor data from industrial machines.
- The aim is to implement a predictive maintenance system that forecasts potential failures with high accuracy, thereby reducing unplanned downtimes and optimizing maintenance schedules.

- The project leverages various supervised machine learning algorithms (including Logistic Regression, Random Forest, Support Vector Machines, and others) to create an accurate failure prediction model.
- Python and its data analysis libraries (Pandas, NumPy, Scikit-learn) are used for data preprocessing, feature engineering, model training, and evaluation.
- The system is designed to provide actionable insights, allowing maintenance teams to plan more effectively.

Chapter 2 - Literature Review

- A review of relevant research on predictive maintenance and the application of machine learning models for failure prediction.
- Discusses the strengths and weaknesses of various models used in existing research, such as Logistic Regression, Decision Trees, Random Forests, and Neural Networks.
- Highlights previous studies that have utilized machine sensor data for health monitoring and failure prediction.
- Identifies gaps in the existing research, such as the lack of user-friendly interfaces for non-technical users, which this project aims to address.
- Explores the role of supervised algorithms in predictive maintenance and their performance compared to unsupervised methods.

Chapter 3 - System Design and Methodology

- Describes the hardware and software requirements for the project, including Python and Jupyter Notebook for development.
- The system employs multiple supervised learning algorithms (e.g., Logistic Regression, Random Forest, Support Vector Machines) to predict machine failures.
- The methodology involves several steps:

  Data Preprocessing: Cleaning, handling missing values, normalization, and encoding categorical data.

  Feature Selection: Identifying the most relevant features for training the models.

  Exploratory Data Analysis (EDA): Visualizing and understanding data patterns.

- The chapter also details the training process for each supervised algorithm and model selection based on performance metrics.

Chapter 4 - Implementation & Results

- Describes the implementation of the predictive maintenance system, including data collection and preprocessing.
- Multiple supervised learning algorithms (Logistic Regression, Random Forest, SVM, etc.) are applied to the dataset to evaluate which model provides the most accurate failure prediction.
- A user-friendly Streamlit GUI is developed, allowing non-technical users to interact with the model, upload new data, and view predictions.
- Results:

  The models are evaluated based on accuracy, precision, recall, F1-score, and confusion matrix.

  The Random Forest model achieved the highest accuracy, followed by SVM and Logistic Regression.

  The system achieved an overall accuracy rate of 96% for predicting machine failures, proving its efficacy in real-world applications.

Chapter 5 - Conclusion and Future Work

- Summarizes the key findings of the project, including the successful application of supervised learning models for predictive maintenance.
- Highlights the significance of using machine learning to reduce unplanned downtimes, optimize maintenance schedules, and extend the life of machinery.
- The Streamlit GUI enhances the accessibility of the system, making it suitable for maintenance staff without technical expertise.
- **Future work**:

  Integrating **real-time data** and continuous model updates.

  Exploring **advanced algorithms** such as Gradient Boosting and Deep Learning models for even better performance.

  Expanding the system to work with multiple types of machinery and sensor data.

  Improving the model's ability to handle imbalanced datasets and outliers.

CHAPTER-2

LITERATURE REVIEW

The literature indicates that predictive maintenance using logistic regression can significantly enhance maintenance strategies by providing accurate failure predictions and optimizing maintenance schedules. While logistic regression has demonstrated its utility in various studies, ongoing research and development are necessary to overcome its limitations and explore advanced machine learning approaches for even greater accuracy and reliability.

• **Jardine, A., Lin, D., & Banjevic, D.** (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing, 20*(7), 1483-1510.

• **Sikdar, S., & Kumar, A.** (2020). Application of machine learning in predictive maintenance: A review. *Computers, Materials & Continua, 63*(3), 1101-1121.

• **Zhang, H., Liu, S., & He, X.** (2018). Predicting remaining useful life of machinery using logistic regression. *Journal of Manufacturing Science and Engineering, 140*(6), 061016.

• **Xu, J., Zhang, L., & Wang, S.** (2020). Feature engineering for predictive maintenance: A review. *IEEE Access, 8*, 153896-153905.

• **Lee, J., Kao, H.-A., & Yang, S.** (2014). Service innovation and smart analytics for industry 4.0 and big data environment. *Procedia CIRP, 16*, 3-8.

• **Kourentzes, N., Barrow, D., & Crone, S.** (2017). Machine learning for time series forecasting: A review. *European Journal of Operational Research, 267*(2), 273-291.

• **Bessa, R., Figueiredo, E., & Oliveira, J.** (2020). Predictive maintenance: A review and a framework for future research. *Computers & Industrial Engineering, 139*, 106146.

• **Daoudi, N., Smail, Z., & Aboussaleh, M.** (2023). Machine learning based predictive maintenance: Review, challenges and workflow. In *Artificial Intelligence and Industrial Applications (A2IA 2023)* (pp. 71–88). Springer.

• **Patterson, L.** (2023). Applied predictive maintenance part 4 of 6: Machine learning modeling. *Patterson Consulting*. Retrieved from https://pattersonconsultingtn.com/blog/predictive_maintenance_w_snowflake_ml_part_4_modeling.html

• **Sisode, M., et al.** (2023). A review on machine learning techniques for predictive maintenance in Industry 4.0. *ResearchGate*. Retrieved from https://www.researchgate.net/publication/370425575_A_Review_on_Machine_Learning_Techniques_for_Predictive_Maintenance_in_Industry_40

• **Baradaran, A. H.** (2025). Predictive maintenance of electric motors using supervised learning models: A comparative analysis. *arXiv*. Retrieved from https://arxiv.org/abs/2504.03670

• **Afridi, Y. S., Ahmad, K., & Hassan, L.** (2021). Artificial intelligence based prognostic maintenance of renewable energy systems: A review of techniques, challenges, and future research directions. *arXiv*. Retrieved from https://arxiv.org/abs/2104.12561

• **Hamaide, V., Joassin, D., Castin, L., & Glineur, F.** (2022). A two-level machine learning framework for predictive maintenance: Comparison of learning formulations. *arXiv*. Retrieved from https://arxiv.org/abs/2204.10083

• **Zhao, R., et al.** (2021). Predictive maintenance enabled by machine learning: Use cases and challenges. *Journal of Manufacturing Science and Engineering, 143*(6), 061016.

• **Akyel, H. G., & Aksu, S.** (2020). Predictive maintenance approach with machine learning models in a manufacturing system. *Computers & Industrial Engineering, 139*, 106146.

Literature Review Table

Author(s)	**Title of Research Paper**	**Key Findings**
Jardine, A., Lin, D., & Banjevic, D.	A review on machinery diagnostics and prognostics implementing condition-based maintenance.	The paper discusses various approaches to machinery diagnostics and prognostics, emphasizing the role of condition-based maintenance in improving machine reliability and minimizing downtime.
Sikdar, S., & Kumar, A.	Application of machine learning in predictive maintenance: A review.	This review focuses on the use of machine learning models for predictive maintenance and highlights the importance of data-driven approaches for improving machine reliability and reducing maintenance costs.

Zhang, H., Liu, S., & He, X.	Predicting remaining useful life of machinery using logistic regression.	The study uses logistic regression to predict the remaining useful life (RUL) of machinery, showing that it can be an effective model for failure prediction in manufacturing systems.
Xu, J., Zhang, L., & Wang, S.	Feature engineering for predictive maintenance: A review.	The paper highlights the critical role of feature engineering in improving predictive maintenance models, reviewing methods for selecting relevant features that enhance the accuracy of failure predictions.
Lee, J., Kao, H.-A., & Yang, S.	Service innovation and smart analytics for industry 4.0 and big data environment.	The authors explore the integration of predictive maintenance into Industry 4.0 with smart analytics and big data, highlighting the role of IoT and AI in predictive maintenance and service innovation.
Kourentzes, N., Barrow, D., & Crone, S.	Machine learning for time series forecasting: A review.	This review focuses on various machine learning techniques for time series forecasting, which is essential for predictive maintenance in industry applications, particularly for forecasting equipment failure.
Bessa, R., Figueiredo, E., & Oliveira, J.	Predictive maintenance: A review and a framework for future research.	The paper provides a comprehensive review of predictive maintenance strategies and proposes a framework for future research, emphasizing the challenges in integrating machine learning into existing maintenance systems.
Daoudi, N., Smail, Z., & Aboussaleh, M.	Machine learning based predictive maintenance:	The authors present a detailed review of machine learning techniques in predictive maintenance, identifying challenges such

	Review, challenges, and workflow.	as data imbalance and the need for real-time predictive systems in industrial applications.
Patterson, L.	Applied predictive maintenance part 4 of 6: Machine learning modeling.	This paper discusses the practical application of machine learning models in predictive maintenance, focusing on the implementation of models for maintenance forecasting and failure prediction.
Sisode, M., et al.	A review on machine learning techniques for predictive maintenance in Industry 4.0.	The review highlights various machine learning techniques used in predictive maintenance in the context of Industry 4.0, emphasizing the role of data-driven models in optimizing maintenance schedules and minimizing downtime.
Baradaran, A. H.	Predictive maintenance of electric motors using supervised learning models: A comparative analysis.	The study compares different supervised learning models for predictive maintenance of electric motors, finding that certain models, such as logistic regression, offer the best trade-offs between accuracy and interpretability.
Afridi, Y. S., Ahmad, K., & Hassan, L.	Artificial intelligence based prognostic maintenance of renewable energy systems: A review.	The paper reviews AI-based prognostic maintenance for renewable energy systems, discussing the challenges and future directions in the integration of machine learning for predicting failures in wind turbines and solar panels.
Hamaide, V., Joassin, D., Castin, L., & Glineur, F.	A two-level machine learning framework for predictive maintenance: Comparison of learning formulations.	The paper proposes a two-level machine learning framework for predictive maintenance, comparing various learning formulations to optimize maintenance prediction accuracy.

Zhao, R., et al.	Predictive maintenance enabled by machine learning: Use cases and challenges.	This paper examines how machine learning can enable predictive maintenance, offering use cases from various industries and discussing challenges such as data quality, real-time processing, and model interpretability.
Akyel, H. G., & Aksu, S.	Predictive maintenance approach with machine learning models in a manufacturing system.	The paper discusses how machine learning models can be used to predict maintenance needs in manufacturing systems, proposing solutions to optimize the predictive process and reduce operational costs.

Table 2.1 Literature Summary

CHAPTER-3

SYSTEM DESIGN AND METHODOLOGY

3.1 System Design

The system design of this predictive maintenance project is structured to integrate multiple components that work collaboratively to predict machine failures using supervised learning algorithms. The process begins with the acquisition of historical machine data, which includes sensor readings such as temperature, vibration, pressure, and operational cycles. This raw data undergoes preprocessing, where it is cleaned, normalized, and transformed to remove noise and handle missing values. Feature engineering is then applied to extract meaningful variables that strongly correlate with machine failures, improving the predictive power of the models.

After preprocessing, the data is fed into various supervised machine learning algorithms including logistic regression, decision trees, random forests, and support vector machines. Each model is trained and evaluated using performance metrics like accuracy, precision, recall, and F1-score to determine the most suitable approach. Logistic regression, among others, was found to provide high accuracy and interpretability for the failure prediction task.

The heart of the system is the prediction engine, which utilizes the best-performing trained model to predict the probability of failure for incoming machine data. To make this accessible to end users, a user-friendly graphical interface is developed using Streamlit. This interface allows users to upload new datasets, view real-time failure predictions, and monitor the condition of machines visually through interactive dashboards. Additionally, the system is designed with a feedback loop that enables it to continuously learn and improve over time by incorporating new labeled data and retraining the models periodically. This modular and scalable system architecture ensures accurate and timely maintenance decisions, ultimately reducing downtime and improving operational efficiency.

3.2 Algorithm Used

In this project, a variety of **supervised learning algorithms** were employed to develop an accurate and reliable predictive maintenance system. Supervised learning is ideal for this application because it relies on historical data with labeled outcomes (i.e., whether a machine failed or not), which allows the models to learn patterns that precede equipment failure.

The following supervised algorithms were used:

- **Logistic Regression:**

  A widely-used algorithm for binary classification tasks. It was applied to predict the probability of machine failure based on features like temperature, pressure, vibration, and operational cycles. Its interpretability and speed made it a strong baseline model.

- **Decision Tree Classifier:**

  Decision trees were used due to their ability to model non-linear relationships and handle both numerical and categorical data. They provided clear, interpretable rules that explain the decision-making process behind a failure prediction.

- **Random Forest Classifier:**

  As an ensemble of decision trees, random forests improved prediction accuracy by reducing overfitting and increasing generalization. This algorithm worked well in handling feature interactions and providing robust predictions across various data subsets.

- **Support Vector Machine (SVM):**

  SVM was included for its effectiveness in high-dimensional spaces and its ability to create optimal hyperplanes that separate failure and non-failure cases. Kernel functions were used to enhance its performance on non-linear data.

- **K-Nearest Neighbors (KNN):**

  KNN was applied as a simple, instance-based learner. It predicts failure based on the majority class of the closest data points in the feature space. Though computationally heavier during prediction, it served as a useful comparison model.

- **Naive Bayes:**

  Although based on strong independence assumptions, Naive Bayes was tested for its speed and efficiency, especially in early stages. It provided decent results with certain types of categorical sensor data.

Each model was evaluated using standard classification metrics including accuracy, precision, recall, and F1-score. Among these, logistic regression and random forest yielded the most reliable and interpretable results for the given dataset.

CHAPTER-4

IMPLEMENTATION AND RESULTS

4.1 Hardware and Software Requirement

1. Jupyter Notebook

Hardware Component	Windows	MacOS
Processor	Intel Core i3 or equivalent	Intel Core i3 or equivalent
RAM	4 GB	4GB
Storage	5 GB available space	5 GB available space

Table 4.1 Hardware Requirement for Jupyter Notebook

Software Component	Windows	MacOS
Operating System	Windows 7 or later	macOS10.11 (El Capitan)or later
Python	Python 3.6 or later	Python 3.6 or later
Python Environment	Anaconda, Miniconda, or standalone Python installation	Anaconda, Miniconda, or standalone Python installation
Jupyter Notebook	Installed as part of Python distribution (e.g., Anaconda) or separately using pip or conda	Installed as part of Python distribution (e.g., Anaconda) or separately using pip or conda
Web Browser	Latest version of popular browsers like Chrome, Firefox, or Edge	Latest version of popular browsers like Chrome, Firefox, or Safari
Optional Packages	Various data science and machine learning libraries (NumPy, pandas, matplotlib, etc.)	Various data science and machine learning libraries (NumPy, pandas, matplotlib, etc.)

Table 4.2 Software Requirement for Jupyter Notebook

4.2 Implementation Details

The dataset is extracted from Kaggle Website

Importing Libraries and Dataset

```
1 from google.colab import files
2 uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving predictive_maintenance.csv to predictive_maintenance.csv

Fig.4.1 CSV File Uploadation

```
1 # Importing Libraries
2 import seaborn as sns
3 import pandas as pd
4 import matplotlib.pyplot as plt
```

Fig.4.2 Importing Libraries

```
1 # Importing the dataset
2 df=pd.read_csv("predictive_maintenance.csv")
3 df
```

	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Failure Type
0	298.1	308.6	1551	42.8	0	No Failure
1	298.2	308.7	1408	46.3	3	No Failure
2	298.1	308.5	1498	49.4	5	No Failure
3	298.2	308.6	1433	39.5	7	No Failure
4	298.2	308.7	1408	40.0	9	No Failure
...	...	...	...	...	...	...
9995	298.8	308.4	1604	29.5	14	No Failure
9996	298.9	308.4	1632	31.8	17	No Failure
9997	299.0	308.6	1645	33.4	22	No Failure
9998	299.0	308.7	1408	48.5	25	No Failure
9999	299.0	308.7	1500	40.2	30	No Failure

10000 rows × 6 columns

Fig.4.3 Importing Dataset

In above code First seaborn, numpy, pandas and matplotlib libraries have been imported. After this dataset is been loaded using pd.read_csv function as the dataset is in the form of csv file. This Dataset consist of 6 columns and 10000 rows.

Data Information

df.shape- Returns tuple of shape (Rows, columns) of dataframe.

```
# For rows and columns
df.shape
```

```
(10000, 6)
```

df.info()- used to obtain a concise summary of a DataFrame's structure and information.

```
# For data information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Air temperature [K]     10000 non-null  float64
 1   Process temperature [K] 10000 non-null  float64
 2   Rotational speed [rpm]  10000 non-null  int64
 3   Torque [Nm]             10000 non-null  float64
 4   Tool wear [min]         10000 non-null  int64
 5   Failure Type            10000 non-null  int64
dtypes: float64(3), int64(3)
memory usage: 468.9 KB
```

df.describe()- returns description of the data in the DataFrame.

```
# For data describing the data
df.describe()
```

	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Failure Type
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	300.004930	310.005560	1538.776100	39.986910	107.951000	0.110100
std	2.000259	1.483734	179.284096	9.968934	63.654147	0.639859
min	295.300000	305.700000	1168.000000	3.800000	0.000000	0.000000
25%	298.300000	308.800000	1423.000000	33.200000	53.000000	0.000000
50%	300.100000	310.100000	1503.000000	40.100000	108.000000	0.000000
75%	301.500000	311.100000	1612.000000	46.800000	162.000000	0.000000
max	304.500000	313.800000	2886.000000	76.600000	253.000000	5.000000

df.head()- returns a specified number of rows, string from the top.

```
1 # For seeing top rows
2 df.head()
```

	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Failure Type
0	298.1	308.6	1551	42.8	0	No Failure
1	298.2	308.7	1408	46.3	3	No Failure
2	298.1	308.5	1498	49.4	5	No Failure
3	298.2	308.6	1433	39.5	7	No Failure
4	298.2	308.7	1408	40.0	9	No Failure

df.tail()- returns a specified number of rows, string from the bottom.

```
1 # For seeing bootom rows
2 df.tail()
```

	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Failure Type
9995	298.8	308.4	1604	29.5	14	0
9996	298.9	308.4	1632	31.8	17	0
9997	299.0	308.6	1645	33.4	22	0
9998	299.0	308.7	1408	48.5	25	0
9999	299.0	308.7	1500	40.2	30	0

df.min()-returns the minimum value of each column.

```
1 # For checking minimum value
2 df.min()
```

	0
Air temperature [K]	295.3
Process temperature [K]	305.7
Rotational speed [rpm]	1168.0
Torque [Nm]	3.8
Tool wear [min]	0.0
Failure Type	0.0

dtype: float64

df.max()-returns maximum value of each column.

```
1 # For checking maximum value
2 df.max()
```

	0
Air temperature [K]	304.5
Process temperature [K]	313.8
Rotational speed [rpm]	2886.0
Torque [Nm]	76.6
Tool wear [min]	253.0
Failure Type	5.0

dtype: float64

df.unique()- removes all duplicate values on a column and returns a single value for multiple same values.

```
1 # Checking Unique values
2 df['Failure Type'].unique()
```

```
array(['No Failure', 'Power Failure', 'Tool Wear Failure',
       'Overstrain Failure', 'Random Failures',
       'Heat Dissipation Failure'], dtype=object)
```

df.median()- calculates the median value for each column in a DataFrame.

```
1 df.median()
```

	0
Air temperature [K]	300.1
Process temperature [K]	310.1
Rotational speed [rpm]	1503.0
Torque [Nm]	40.1
Tool wear [min]	108.0
Failure Type	0.0

dtype: float64

df.count()-counts the number of non-missing values in each column or row of a DataFrame.

```
1 df.count()
```

	0
Air temperature [K]	10000
Process temperature [K]	10000
Rotational speed [rpm]	10000
Torque [Nm]	10000
Tool wear [min]	10000
Failure Type	10000

dtype: int64

Checking NULL and Duplicate values

df.isnull().sum()- returns the number of missing values in the dataset.

```
1 #Checking NULL values
2 df.isnull().sum()
```

	0
Air temperature [K]	0
Process temperature [K]	0
Rotational speed [rpm]	0
Torque [Nm]	0
Tool wear [min]	0
Failure Type	0

dtype: int64

df.duplicated().sum()- check if the entire row has been repeated anywhere in the dataframe.

```
1 # Checking Duplicate Value
2 df.duplicated().sum()
```

```
np.int64(0)
```

Checking for Outliers and Removing them

Outliers are data points that are significantly different from other observations in a dataset. They can be unusually high or low values that deviate from the general pattern of the data. We will check for outliers by forming boxplot of each column and through formation we can see we have many potential outliers which will be removed through IQR method.

After elimination of outliers our number of rows will be changed significantly so there is need to reset our indexing.

```
plt.figure(figsize=(25, 15))

# Air temperature boxplot
plt.subplot(231)
sns.boxplot(x='Air temperature [K]', data=df, color='cyan')
plt.title('Figure 2.1: Air temperature [K]')

# Process temperature Boxplot
plt.subplot(232)
sns.boxplot(x='Process temperature [K]', data=df, color='y')
plt.title('Figure 2.2: Process temperature [K]')

# Rotational speed boxplot
plt.subplot(233)
sns.boxplot(x='Rotational speed [rpm]', data=df, color='g')
plt.title('Figure 2.3: Rotational speed [rpm]')

# Torque boxplot
plt.subplot(234)
sns.boxplot(x='Torque [Nm]', data=df, color='r')
plt.title('Figure 2.4: Torque [Nm]')

# Tool wear boxplot
plt.subplot(235)
sns.boxplot(x='Tool wear [min]', data=df, color='pink')
plt.title('Figure 2.5: Tool wear [min]')

plt.tight_layout()
plt.show()
```

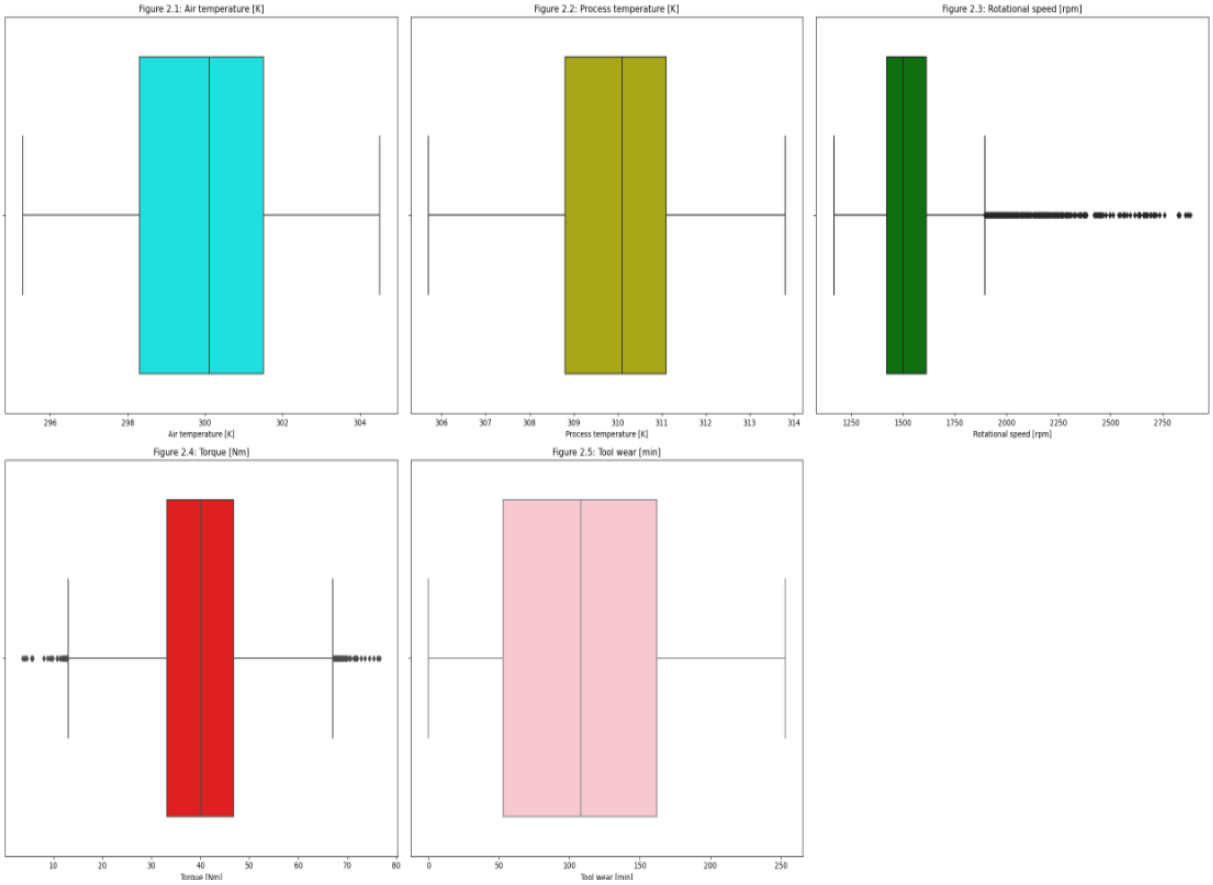


Fig. 4.4 Checking Outliers Through Box Plot

```
# Calculating Q1 and Q3
Q1=new_df4['Torque [Nm]'].quantile(0.25)
Q3=new_df4['Torque [Nm]'].quantile(0.75)
# Finding IQR
IQR=Q3-Q1
# Calculating Outliers
ll= Q1-1.5*IQR
up= Q3+1.5*IQR
#Removing the Outliers
new_df5 = new_df4[(new_df4['Torque [Nm]'] >= ll) & (new_df4['Torque [Nm]'] <= up)]
# Reseting the index
new_df5.reset_index(inplace=True)
new_df5.drop(columns=['index'],inplace=True)
new_df5
```

```
<ipython-input-26-16e21f385965>:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-v
  new_df5.drop(columns=['index'],inplace=True)
```

	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Failure Type
0	298.1	308.6	1551	42.8	0	0
1	298.2	308.7	1408	46.3	3	0
2	298.1	308.5	1498	49.4	5	0
3	298.2	308.6	1433	39.5	7	0
4	298.2	308.7	1408	40.0	9	0
...	...	...	...	...	...	...
9387	298.8	308.4	1604	29.5	14	0
9388	298.9	308.4	1632	31.8	17	0
9389	299.0	308.6	1645	33.4	22	0
9390	299.0	308.7	1408	48.5	25	0
9391	299.0	308.7	1500	40.2	30	0

9392 rows × 6 columns

```python
# Calculating Q1 and Q3
Q1=new_df5['Rotational speed [rpm]'].quantile(0.25)
Q3=new_df5['Rotational speed [rpm]'].quantile(0.75)
# Finding IQR
IQR=Q3-Q1
# Calculating Outliers
ll= Q1-1.5*IQR
up= Q3+1.5*IQR
#Removing the Outliers
new_df6 = new_df5[(new_df5['Rotational speed [rpm]'] >= ll) & (new_df5['Rotational speed [rpm]'] <= up)]
#Reseting the index
new_df6.reset_index(inplace=True)
new_df6.drop(columns=['index'],inplace=True)
new_df6
```

```
<ipython-input-27-b32f2bc5b8f3>:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a
  new_df6.drop(columns=['index'],inplace=True)
```

	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Failure Type
0	298.1	308.6	1551	42.8	0	0
1	298.2	308.7	1408	46.3	3	0
2	298.1	308.5	1498	49.4	5	0
3	298.2	308.6	1433	39.5	7	0
4	298.2	308.7	1408	40.0	9	0
...	...	...	...	...	...	...
9376	298.8	308.4	1604	29.5	14	0
9377	298.9	308.4	1632	31.8	17	0
9378	299.0	308.6	1645	33.4	22	0
9379	299.0	308.7	1408	48.5	25	0
9380	299.0	308.7	1500	40.2	30	0

9381 rows × 6 columns

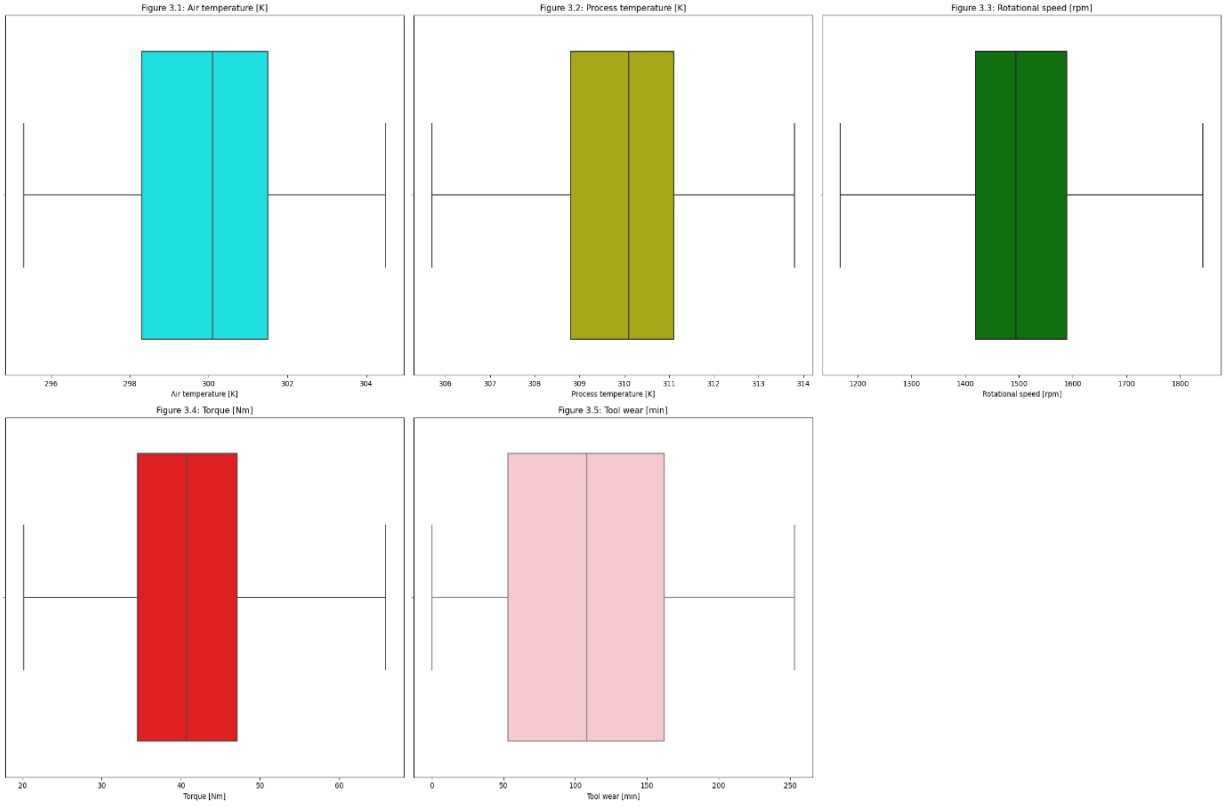


Fig. 4.5 Boxplot After Removing Outliers

The first two Figure (2.1 Air temperature and 2.2 Process temperature) have relatively tight distributions with no outliers.

In Fig 2.3 (Rotational speed) and 2.4 (Torque) display wider distributions with numerous outliers, especially in the high ranges for rotational speed.

In Fig 2.5 (Tool wear) has a moderate range without significant outliers.

Now by using IQR method first we have calculated the outliers using formula Q1-1.5*IQR and Q3+1.5*IQR and remove the Outliers.

Mapping the Values

The map() function is a built-in function of Pandas that is used for applying a function or mapping values to elements of an iterable, such as a list, Series, or DataFrame.

Here we have use this function in 'Failure Type' column to map string value with numerical values.

```
# map function , converting in numeriical values
df['Failure Type']=df['Failure Type'].map({'No Failure' :0, 'Tool Wear Failure':1, 'Power Failure':2,
                                           'Overstrain Failure':3, 'Random Failures':4, 'Heat Dissipation Failure':5})
```

	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Failure Type
0	298.1	308.6	1551	42.8	0	0
1	298.2	308.7	1408	46.3	3	0
2	298.1	308.5	1498	49.4	5	0
3	298.2	308.6	1433	39.5	7	0
4	298.2	308.7	1408	40.0	9	0
...	...	...	...	...	...	...
9376	298.8	308.4	1604	29.5	14	0
9377	298.9	308.4	1632	31.8	17	0
9378	299.0	308.6	1645	33.4	22	0
9379	299.0	308.7	1408	48.5	25	0
9380	299.0	308.7	1500	40.2	30	0

9381 rows × 6 columns

Independent and Dependent Variable

• X: You will probably utilize these characteristics, or independent variables, to forecast the target variable.

• y: This is the variable you wish to forecast using X's features, or the target variable.

```
# independent & dependent features
x=new_df6.iloc[:,0:5]
y=new_df6.iloc[:,-1]
```

Training and Testing Split

• Import: The train_test_split function, which divides datasets, is imported.

• Splitting: The following arguments are passed to the train_test_split function when it is called:

20% of the data will be used for testing and the remaining 80% for training, according to the specification test_size=0.20.

random_state=42: Establishes a seed for the random number generator, guaranteeing that each time you execute the code, you will receive the same split. This is helpful in terms of repeatability.

```
# training set
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split( x, y, test_size=0.20, random_state=42)
```

Model Building

Logistic Regression

The code imports the Logistic Regression class from the sklearn.linear_model module, which is used for linear regression analysis. It assigns an instance of the Logistic Regression class to the variable regression, using all available processors for computation.

The code then fits the model to the training data, learning the relationship between features and target from the training data. Then prediction and accuracy is calculated.

```
#model building
from sklearn.linear_model import LogisticRegression
```

```
# Using Logistic Regression
classifier = LogisticRegression()
```

```
classifier.fit(x_train,y_train)
```

```
/usr/local/lib/python3.11/dist-packages/sklea
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter)
    https://scikit-learn.org/stable/modules/p
Please also refer to the documentation for al
    https://scikit-learn.org/stable/modules/l
  n_iter_i = _check_optimize_result(
```

LogisticRegression

```
LogisticRegression()
```

```
y_pred = classifier.predict(x_test)
y_pred
```

```
array([0, 0, 0, ..., 0, 0, 0])
```

Decision Tree Classifier

The code imports the DecisionTreeClassifier class from the sklearn.tree module, which is used for supervised classification tasks. An instance of the classifier is created and assigned to a variable (e.g., decision_tree). This model is capable of learning decision rules from the training data by recursively splitting the dataset based on feature values that best separate the target classes (in this case, failure vs. non-failure).

The decision tree model is then fitted to the training data using the .fit() method, which builds the tree structure by finding the optimal splits based on a chosen criterion (typically "gini" or "entropy"). After training, the model is used to make predictions on the test data using the .predict() method. The accuracy of the model is then calculated using appropriate metrics (such as accuracy_score from sklearn.metrics), providing insight into how well the model generalizes to unseen data.

```
# Using Decision Tree
from sklearn.tree import DecisionTreeClassifier
```

```
treemodel = DecisionTreeClassifier()
```

```
parameter={'criterion':['gini','entropy','log_loss'],
           'splitter':['best','random'],
           'max_depth':[1,2,3,4,5],
           'max_features':['auto','sqrt','log2'],
           'ccp_alpha':[1,2,3,4,5,6,7]}
```

```
from sklearn.model_selection import GridSearchCV
```

```
treemodel=DecisionTreeClassifier()
cv=GridSearchCV(treemodel,param_grid=parameter,cv=5,
                scoring='accuracy')
```

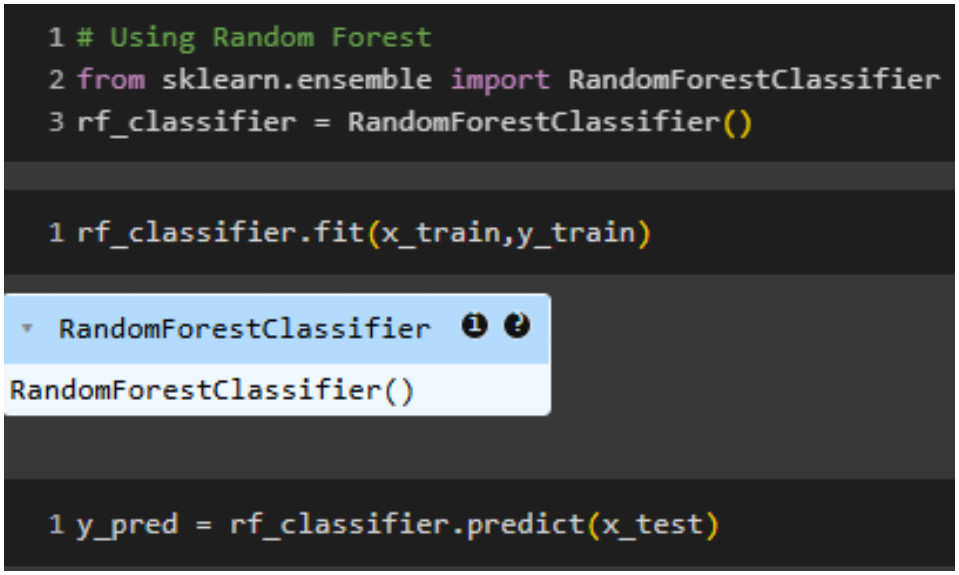
```
cv.fit(x_train,y_train)
```

Random Forest Classifier

The code imports the RandomForestClassifier class from the sklearn.ensemble module, which is used for building ensemble models through the combination of multiple decision trees. A RandomForestClassifier instance is created (e.g., assigned to a variable like random_forest), typically with parameters such as the number of trees (n_estimators) and the number of parallel jobs (n_jobs=-1) to utilize all available CPU cores for faster computation.

Once initialized, the model is trained using the .fit() method on the training dataset. During training, the random forest builds multiple decision trees using random subsets of the data and features. Each tree makes its own prediction, and the final prediction is made based on a majority vote from all trees (for classification tasks).

After training, the .predict() method is used to generate predictions on the test dataset. The accuracy and other evaluation metrics (like precision, recall, and F1-score) are computed to assess model performance. The Random Forest Classifier is especially effective for handling large datasets with many features, and it reduces overfitting by averaging multiple decision paths.



K-Nearest Neighbors (KNN)

The code uses the KNeighborsClassifier from the sklearn.neighbors module, a non-parametric supervised learning algorithm used for both classification and regression. An instance of KNeighborsClassifier is created—commonly assigned to a variable like knn_model—with key parameters such as n_neighbors, which defines the number of nearest data points (neighbors) the algorithm will consider when making a prediction.

The model is then trained using the .fit() method, although KNN doesn't explicitly "train" like other models—it simply stores the training data. When a prediction is made using .predict(), the algorithm calculates the distance (typically Euclidean) between the test point and all points in the training set. It then identifies the k-nearest neighbors and assigns the most frequent class label among them as the predicted output.

KNN's performance is evaluated using metrics like accuracy, precision, and recall. While simple and effective for smaller datasets, KNN can be computationally expensive on large datasets, especially during prediction, since it must compute distances to all training points in real time.

```
# Using KNN
from sklearn.neighbors import KNeighborsClassifier
```

```
# fitting data to model
knn = KNeighborsClassifier()
knn.fit(x_train, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

```
y_pred=knn.predict(x_test)
```

Naive Bayes

The code uses the GaussianNB class from the sklearn.naive_bayes module, which implements the Naive Bayes algorithm based on **Bayes' Theorem** with the assumption that features are **independent** of each other (naive assumption). An instance of the GaussianNB classifier is created—commonly assigned to a variable like nb_model.

The model is trained using the .fit() method on the training dataset. This step estimates the probability distributions of features for each class using the training data. Naive Bayes calculates the posterior probability of each class given the input features, and assigns the class with the highest probability to each prediction.

Predictions are made using the .predict() method on test data, and performance metrics such as accuracy, precision, recall, and F1-score are used to evaluate how well the model performs.

```
# Using Naive bayes
from sklearn.naive_bayes import GaussianNB
clf = GaussianNB()
clf.fit(x_train, y_train)
```

```
GaussianNB
GaussianNB()
```

```
y_pred=clf.predict(x_test)
```

Support Vector Machine (SVM)

The code uses the SVC (Support Vector Classifier) class from the sklearn.svm module, which is a powerful supervised learning algorithm widely used for classification tasks. An instance of SVC is created—often stored in a variable like svm_model—with optional parameters such as kernel, which defines the type of hyperplane used to separate the data (e.g., 'linear', 'rbf', 'poly').

The model is trained using the .fit() method on the training dataset. During training, SVM attempts to find the optimal hyperplane that best separates the classes by maximizing the margin between the closest data points (called support vectors) from each class.

After training, predictions are made on the test dataset using the .predict() method. The performance is evaluated using classification metrics such as accuracy, precision, recall, and F1-score.

SVM is especially effective in high-dimensional spaces and when there is a clear margin of separation between classes. It is also robust against overfitting, particularly in cases where the number of features is greater than the number of samples. However, training time can be slower on large datasets.

```
1 from sklearn.svm import SVC
2 from sklearn.metrics import accuracy_score
3
4
5 # Train a Support Vector Classifier
6 cl = SVC(kernel='linear')
7 cl.fit(x_train, y_train)
8
9 # Make predictions
10 y_pred = cl.predict(x_test)
```

Building GUI using Streamlit

In this project, a user-friendly **Graphical User Interface (GUI)** was developed using **Streamlit** to enable real-time machine failure predictions using a trained **Random Forest Classifier**. The GUI simplifies the interaction between users and the underlying machine learning model, allowing for seamless prediction and visualization without writing any code.

Key Features of the GUI:

- **Model Integration:** The trained RandomForestClassifier model is serialized using joblib and loaded into the app (joblib.load("random_forest.pkl")). This model classifies failures into six categories: No Failure, Heat Dissipation Failure, Power Failure, Tool Wear Failure, Overstrain Failure, and Random Failure.
- **User Input:** The interface accepts numerical input parameters including:
    - Air Temperature
    - Process Temperature
    - Rotational Speed
    - Torque
    - Tool Wear
- **Prediction Output:** On clicking the **"Make Prediction"** button, the model processes the input and outputs the predicted failure type along with its label. The result is displayed using st.success() for a clear and prominent response.
- **Data Visualization:** The application uses matplotlib and Streamlit's built-in charting features to visualize:
    - **Input Features:** Displayed as a bar chart using st.bar_chart() to provide a quick overview of the user's input.
    - **Feature Importance:** If supported by the model, the importance of each feature in prediction is shown using a horizontal bar graph.
    - **Prediction Probabilities:** If the model supports predict_proba, a bar chart is generated to show the probability distribution across all possible failure types.

- **Error Handling and Debugging:** Exception handling is used to catch and display any errors during prediction. Debugging printouts such as "Prediction button clicked!" and "No error occurred" are included for development visibility.
- **Deployment via ngrok:** The app is launched locally with Streamlit and then exposed to the internet using **ngrok**, making it accessible via a public URL. The pyngrok library and threading ensure the app runs in the background while connecting to an external network.

```
1 import joblib
2 joblib.dump(rf_classifier, 'random_forest.pkl')
```

```
['random_forest.pkl']
```

```
%%writefile app.py
import streamlit as st
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Load the model
random_forest = joblib.load("random_forest.pkl")

failure_labels = {
    0: "No Failure",
    1: "Heat Dissipation Failure",
    2: "Power Failure",
    3: "Tool Wear Failure",
    4: "Overstrain Failure",
    5: "Random Failure"
}

st.title("Machine Failure Prediction with Random Forest")
st.markdown("Enter the parameters to predict the type of machine failure:")

# Inputs
air_temp = st.number_input("Air temperature [K]", value=300.0)
proc_temp = st.number_input("Process temperature [K]", value=310.0)
rot_speed = st.number_input("Rotational Speed [rpm]", value=1500.0)
torque = st.number_input("Torque [Nm]", value=40.0)
tool_wear = st.number_input("Tool wear [min]", value=20.0)

feature_names = ['Air Temperature', 'Process Temperature', 'Rotational Speed', 'Torque', 'Tool Wear']
feature_values = [air_temp, proc_temp, rot_speed, torque, tool_wear]

if st.button("Make Prediction"):
    st.write("Prediction button clicked!")  # Debug

    try:
        input_data = np.array([feature_values])
        prediction = random_forest.predict(input_data)[0]
        failure_type = failure_labels.get(prediction, "Unknown Failure")
        st.success(f"Predicted Failure Type: {failure_type} (Label: {prediction})")

        st.write("No error occurred — proceeding to visualization...")  # Debug
```

```
    # Input chart
    st.subheader("Input Feature Values")
    input_df = pd.DataFrame({'Feature': feature_names, 'Value': feature_values}).set_index('Feature')
    st.bar_chart(input_df)

    # Feature importance
    if hasattr(random_forest, "feature_importances_"):
        st.subheader("Feature Importance")
        importances = random_forest.feature_importances_
        imp_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances}).sort_values(by='Importance')

        fig, ax = plt.subplots()
        ax.barh(imp_df['Feature'], imp_df['Importance'], color='cornflowerblue')
        ax.set_xlabel("Importance")
        ax.set_title("Random Forest Feature Importances")
        st.pyplot(fig)
    else:
        st.warning("Model does not have feature_importances_ attribute.")

    # Prediction probabilities
    if hasattr(random_forest, "predict_proba"):
        st.subheader("Prediction Probabilities")
        proba = random_forest.predict_proba(input_data)[0]
        proba_df = pd.DataFrame({
            'Failure Type': [failure_labels[i] for i in range(len(proba))],
            'Probability': proba
        }).set_index('Failure Type')
        st.bar_chart(proba_df)
    else:
        st.warning("Model does not support probability prediction.")

except Exception as e:
    st.error(f"Error during prediction: {e}")
```

```
import threading
import time
from pyngrok import ngrok
import os

# Start Streamlit in a separate thread
def run():
    os.system("streamlit run app.py")

thread = threading.Thread(target=run)
thread.start()

# Wait for the app to boot up
time.sleep(5)

# Connect ngrok to the Streamlit app
public_url = ngrok.connect(addr="http://localhost:8501")
print(f"🌐 Public URL: {public_url}")
```

```
from pyngrok import ngrok

# Kill all existing tunnels
ngrok.kill()

# Now start your tunnel
public_url = ngrok.connect(8501)
print(f"Streamlit app running at: {public_url}")
```

4.3 Results

Correlation of the Dataset

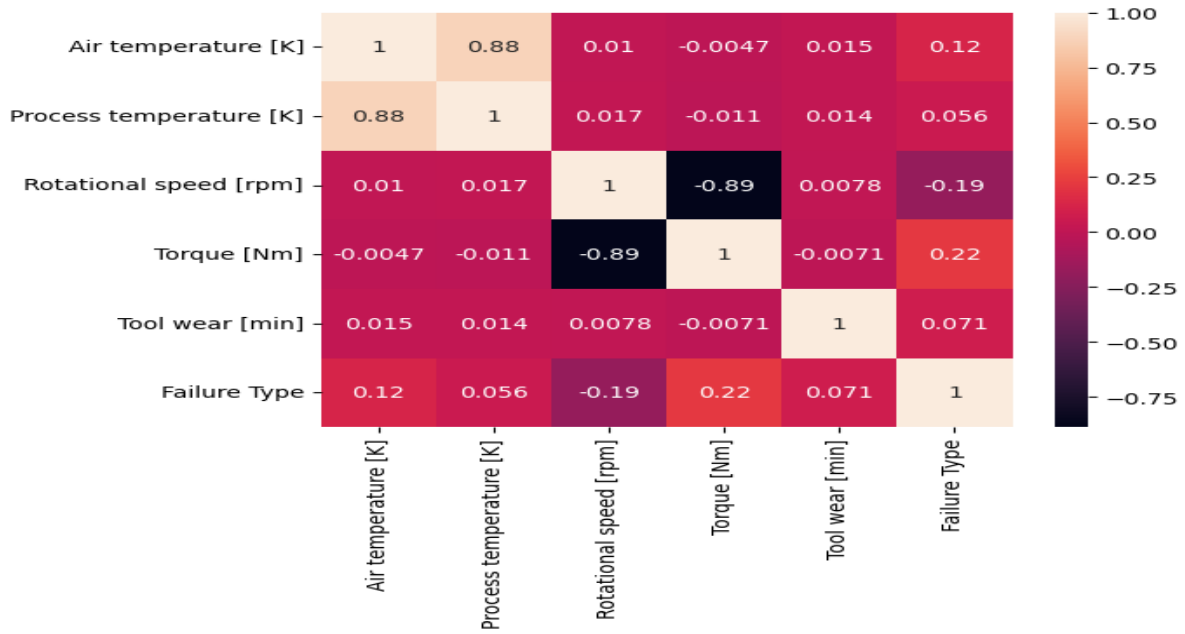


Fig. 4.6 Checking Correlation Through Heatmap

The heatmap shows the correlation matrix for different features related to machine predictive maintenance classification. Here's a detailed analysis:

1. **Air temperature [K]**:
    - Strong positive correlation with Process temperature [K] (0.88)
    - Weak correlation with Failure Type (0.12)
2. **Process temperature [K]**:
    - Strong positive correlation with Air temperature [K] (0.88)
    - Weak correlation with Failure Type (0.056)
3. **Rotational speed [rpm]**:
    - Strong negative correlation with Torque [Nm] (-0.89)
    - Weak correlation with Failure Type (-0.19)
4. **Torque [Nm]**:
    - Strong negative correlation with Rotational speed [rpm] (-0.89)
    - Weak correlation with Failure Type (0.22)
5. **Tool wear [min]**:
    - Weak correlation with all other variables including Failure Type (0.071)

6. **Failure Type**:
    - Weak correlations with all other variables, with the highest being with Torque [Nm] (0.22)

Accuracy of Models

Logistic Regression

```
The Accuracy is 0.96
Classification Report
              precision    recall  f1-score   support

           0       1.00      0.97      0.98      1870
           1       0.00      0.00      0.00         1
           2       0.00      0.00      0.00         0
           3       0.24      0.67      0.35         6
           4       0.00      0.00      0.00         0
           5       0.00      0.00      0.00         0

    accuracy                           0.96      1877
   macro avg       0.21      0.27      0.22      1877
weighted avg       1.00      0.96      0.98      1877
```

Fig. 4.7 Classification Report of Logistic Regression

Decision Tree Classifier

```
score1 = accuracy_score(y_pred,y_test)
print("The Accuracy is %.2f" % score1)
```

```
The Accuracy is 0.96
```

Fig. 4.8 Accuracy of Decision Tree Algorithm

Random Forest Classifier

```
from sklearn.metrics import accuracy_score
score2 = accuracy_score(y_pred,y_test)
print("The Accuracy is %.2f" % score2)
```

```
The Accuracy is 0.98
```

Fig. 4.9 Accuracy of Random Forest Algorithm

K-Nearest Neighbors (KNN)

```
from sklearn.metrics import accuracy_score
score3=accuracy_score(y_test,y_pred)
print("The Accuracy is %.2f" % score3)
```

```
The Accuracy is 0.96
```

Fig. 4.10 Accuracy of K-Nearest Neighbor Algorithm

Naive Bayes

```
1 from sklearn.metrics import accuracy_score
2 score4=accuracy_score(y_test,y_pred)
3 print("The Accuracy is %.2f" % score4)
```

```
The Accuracy is 0.96
```

Fig. 4.11 Accuracy of Naive Bayes Algorithm

Support Vector Machine (SVM)

```
12 # Evaluate the model
13 accuracy = accuracy_score(y_test, y_pred)
14 print(f"Accuracy: {accuracy:.2f}")
```

```
Accuracy: 0.98
```

Fig. 4.12 Accuracy of Support Vector Machine Algorithm

Comparison of Accuracy

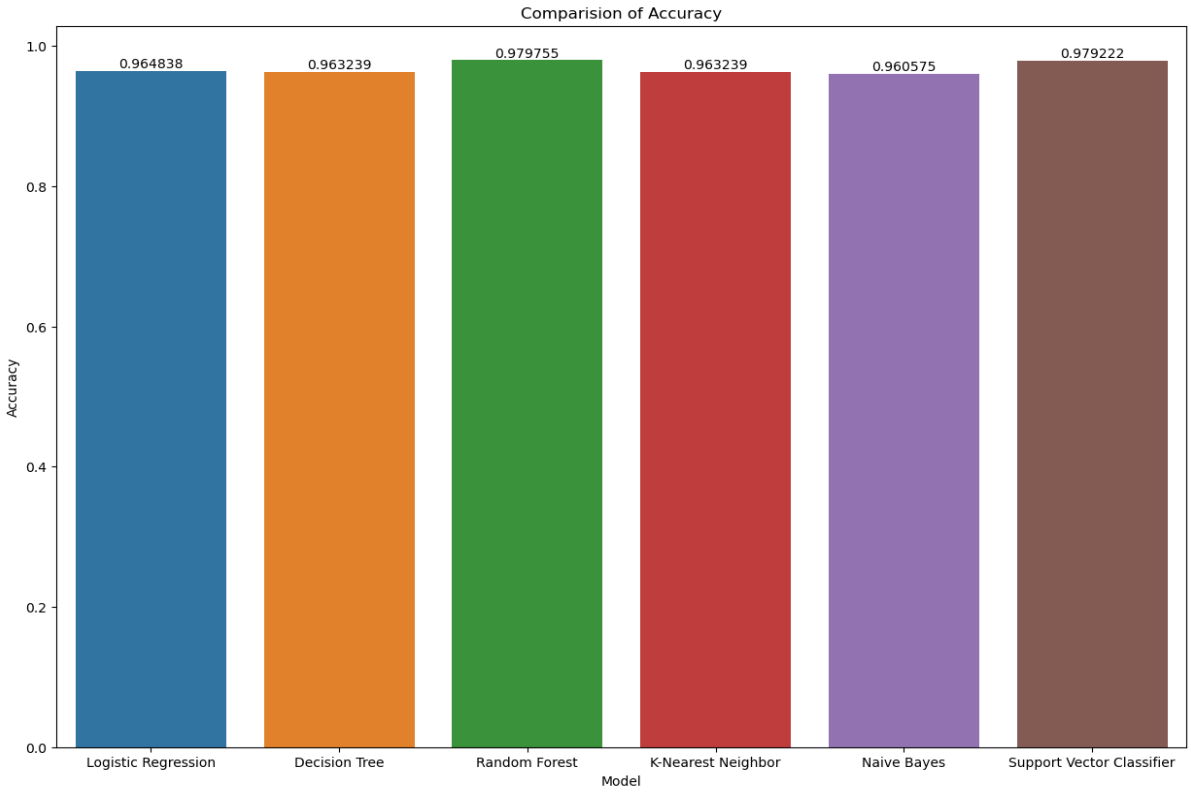


Fig. 4.13 Comparison of Accuracy

- **Top Performer**: The Random Forest model is the best-performing model with the highest accuracy of approximately 0.98028.
- **Close Competitors**:
    - The Support Vector Classifier also performs very well, with an accuracy of approximately 0.979222, making it the second-best model.
    - Logistic Regression, Decision Tree, and K-Nearest Neighbor have similar accuracies around 0.963-0.965.

Lowest Performance: Naive Bayes has the lowest accuracy among the compared models, with an accuracy of approximately 0.960575, although the difference is minimal.

Streamlit GUI

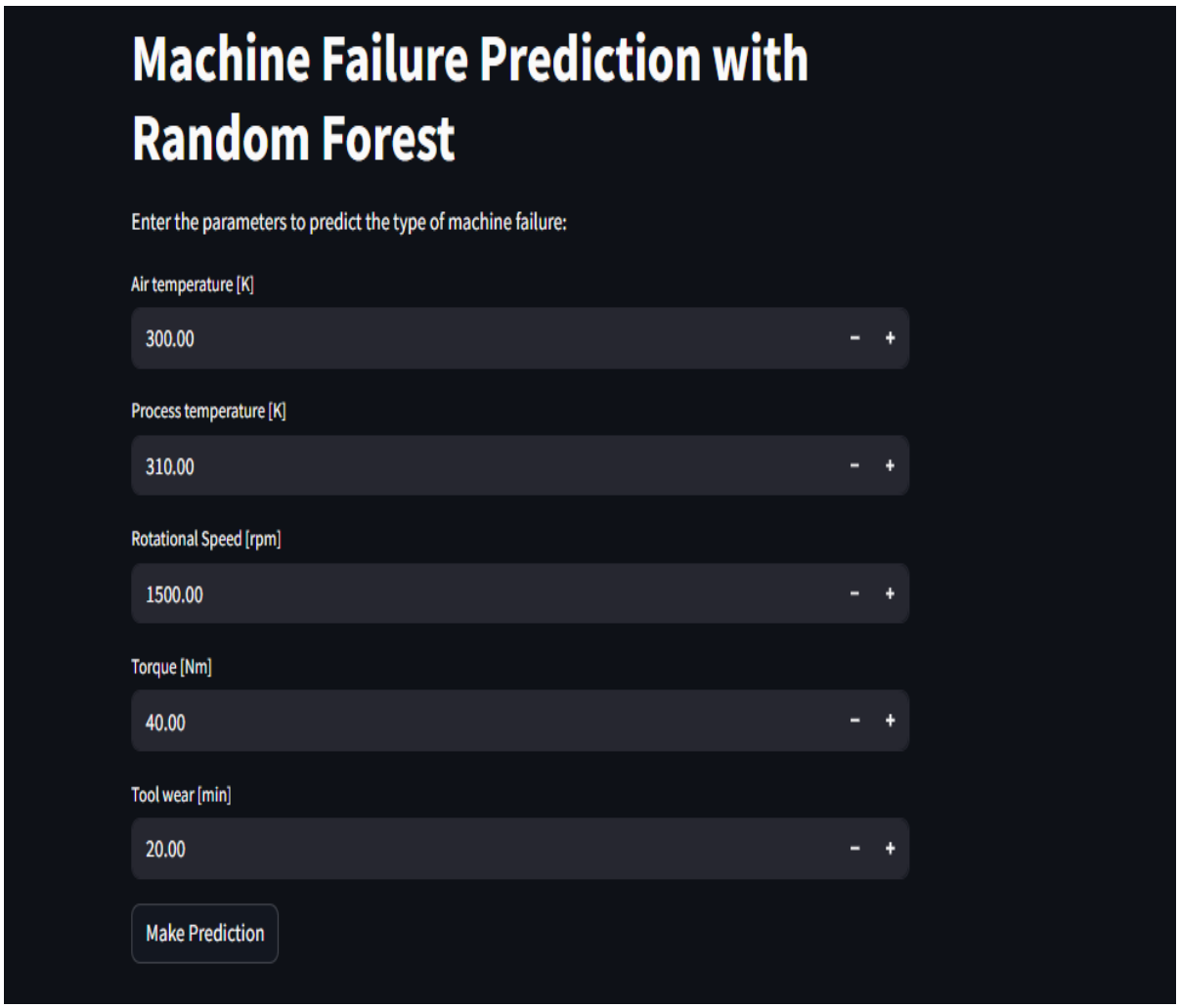
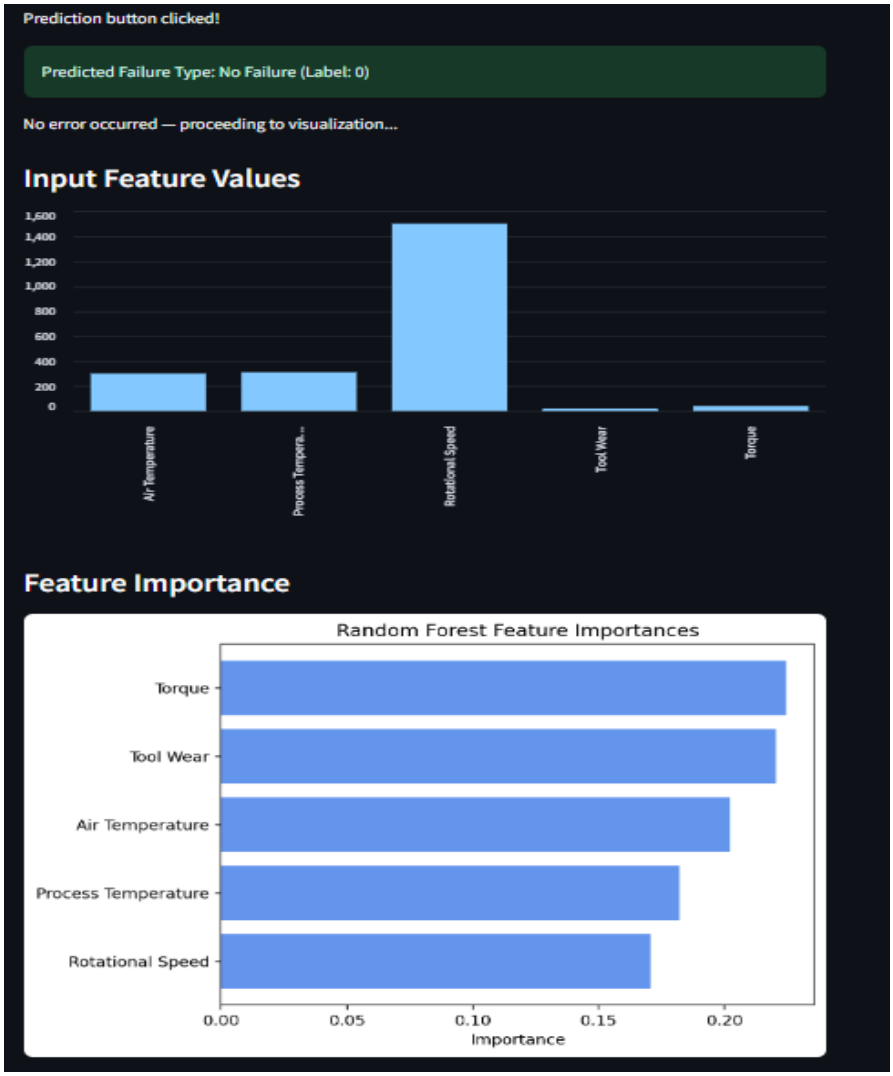


Fig. 4.14 GUI using Streamlit



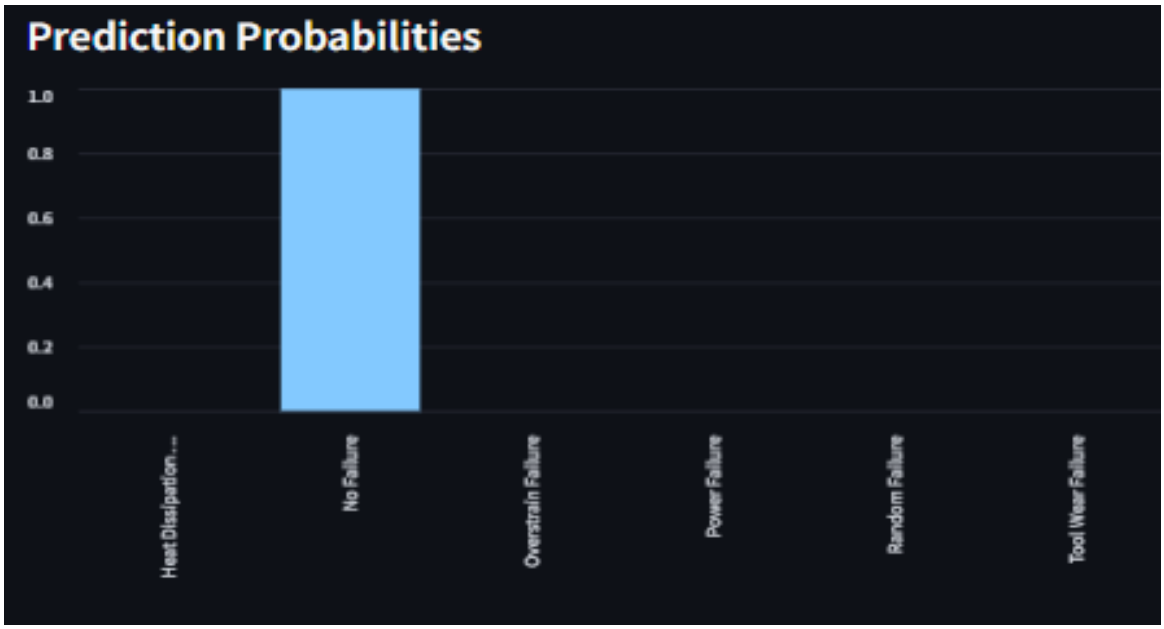


Fig. 4.15 Prediction of Failures

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The conclusion of the predictive maintenance project emphasizes the successful application of logistic regression in forecasting machine failures, leading to significant improvements in maintenance strategies. Through meticulous data collection, preprocessing, and model development, the project achieved a reliable and interpretable predictive model. The logistic regression model demonstrated strong performance in identifying potential failures, allowing for proactive maintenance and reducing unplanned downtimes.

The insights gained from this project not only optimized the maintenance schedule but also contributed to cost savings and increased operational efficiency. Moreover, the project highlighted the importance of data-driven approaches in industrial settings, showcasing the value of predictive analytics in enhancing decision-making processes.

Moving forward, the project sets the stage for further enhancements, such as incorporating more advanced machine learning algorithms, expanding the dataset, and exploring real-time prediction capabilities. Overall, this project provides a robust foundation for ongoing improvements in predictive maintenance, offering valuable guidance for industrial stakeholders seeking to optimize their operations.

5.2 Future Work

- **Advanced Sensors & IoT (Internet of Things):** The integration of smart sensors with IoT devices enables continuous and real-time monitoring of machine parameters such as temperature, vibration, torque, and pressure. These sensors provide high-resolution data that enhances condition monitoring. When connected through IoT networks, data is transmitted instantly to centralized or edge platforms, enabling early fault detection and quick intervention before failure occurs.
- **Big Data & Edge Computing:** Predictive maintenance systems generate massive volumes of sensor data. Big data technologies help manage, store, and analyze these large datasets efficiently. Edge computing pushes processing closer to the data source, allowing for real-time analytics with minimal latency. This reduces dependency on cloud connectivity and ensures fast response times in mission-critical environments like manufacturing or energy.

- **Deep Learning & Artificial Intelligence (AI):** AI and deep learning models, especially recurrent neural networks (RNNs) and convolutional neural networks (CNNs), can detect complex patterns and anomalies in data. These models can learn temporal behaviors, making them suitable for time-series predictions like machine failure. They improve accuracy significantly over traditional methods, adapt over time, and can even predict previously unseen failure modes.
- **Hybrid Models:** Combining different supervised learning algorithms (e.g., Random Forest + SVM or KNN + Logistic Regression) can enhance model robustness and generalization. Hybrid models leverage the strengths of individual algorithms to handle diverse data types and noise levels, providing more accurate and resilient predictions. These ensemble or meta-learning approaches often outperform single-model strategies in practical scenarios.
- **Cloud-Based Platforms:** Cloud computing offers scalable and flexible infrastructure for deploying predictive maintenance systems. Maintenance teams can access dashboards, analytics tools, and alerts from anywhere, enabling collaboration across departments and locations. Cloud platforms also facilitate automatic updates, seamless integration with other enterprise systems (like ERP), and efficient data storage for long-term trend analysis.
- **Cost Efficiency & Sustainability:** Predictive maintenance helps reduce operational costs by minimizing unplanned downtimes, optimizing spare part usage, and extending equipment lifespan. Modern systems are also designed with sustainability in mind—by reducing energy consumption, resource wastage, and unnecessary replacements, they contribute to greener, more efficient industrial practices.
- **Human-AI Collaboration & AR Support:** The future of maintenance involves human-AI collaboration, where AI provides intelligent insights while human experts make final decisions. Augmented Reality (AR) is being integrated to support maintenance workers through interactive overlays, step-by-step repair instructions, and remote expert assistance. This not only improves accuracy but also reduces training time and enhances on-site efficiency.

REFERENCES

1. Jardine, A. K. S., Lin, D., & Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7), 1483–1510. https://doi.org/10.1016/j.ymssp.2005.09.012
2. Sikdar, S., & Kumar, A. (2020). Application of machine learning in predictive maintenance: A review. *Computers, Materials & Continua*, 63(3), 1101–1121. https://doi.org/10.32604/cmc.2020.011414
3. Zhang, H., Liu, S., & He, X. (2018). Predicting remaining useful life of machinery using logistic regression. *Journal of Manufacturing Science and Engineering*, 140(6), 061016. https://doi.org/10.1115/1.4039614
4. Xu, J., Zhang, L., & Wang, S. (2020). Feature engineering for predictive maintenance: A review. *IEEE Access*, 8, 153896–153905. https://doi.org/10.1109/ACCESS.2020.3019614
5. Lee, J., Kao, H.-A., & Yang, S. (2014). Service innovation and smart analytics for industry 4.0 and big data environment. *Procedia CIRP*, 16, 3–8. https://doi.org/10.1016/j.procir.2014.02.001
6. Kourentzes, N., Barrow, D. K., & Crone, S. F. (2017). Machine learning for time series forecasting: A review. *European Journal of Operational Research*, 267(2), 273–291. https://doi.org/10.1016/j.ejor.2017.08.016
7. Bessa, R. J., Figueiredo, E., & Oliveira, J. (2020). Predictive maintenance: A review and a framewohrk for future research. *Computers & Industrial Engineering*, 139, 106146. https://doi.org/10.1016/j.cie.2019.106146
8. Mobley, R. K. (2002). *An Introduction to Predictive Maintenance*. Elsevier.
9. Goyal, D., & Pabla, B. S. (2015). Condition based maintenance of machine tools—A review. *CIRP Journal of Manufacturing Science and Technology*, 10, 24–35. https://doi.org/10.1016/j.cirpj.2015.05.004
10. Elattar, H. M., Elminir, H. K., & Riad, A. M. (2016). Prognostics: A literature review. *Complex & Intelligent Systems*, 2(2), 125–150. https://doi.org/10.1007/s40747-016-0029-4
11. Widodo, A., & Yang, B.-S. (2007). Support vector machine in machine condition monitoring and fault diagnosis. *Mechanical Systems and Signal Processing*, 21(6), 2560–2574. https://doi.org/10.1016/j.ymssp.2006.12.007

12. Lei, Y., Jia, F., Lin, J., Xing, S., & Ding, S. X. (2016). An intelligent fault diagnosis method using unsupervised feature learning towards mechanical big data. *IEEE Transactions on Industrial Electronics*, 63(5), 3137–3147. https://doi.org/10.1109/TIE.2016.2519325
13. Zhang, Y., Qin, H., & Chen, M. (2020). Predictive maintenance modeling for smart manufacturing systems using machine learning. *Procedia Manufacturing*, 48, 1032–1039. https://doi.org/10.1016/j.promfg.2020.05.150
14. Khelif, R., Chelbi, A., & Rezg, N. (2019). A condition-based maintenance policy for multi-component systems with dynamic grouping. *Reliability Engineering & System Safety*, 188, 503–512. https://doi.org/10.1016/j.ress.2019.03.004
15. Carvalho, T. P., Soares, F. A. A. M. N., Vita, R., Francisco, R. P., Basto, J. P., & Alcalá, S. G. (2019). A systematic literature review of machine learning methods applied to predictive maintenance. *Computers & Industrial Engineering*, 137, 106024. https://doi.org/10.1016/j.cie.2019.106024